

SMT Subject:

Robot Escape with Bounded Model Checking

Total: 10 points

1 Context

You are given a Python project named *Robot Escape*. A robot moves on a rectangular grid. Each cell is identified by integer coordinates (x, y) , with $(0, 0)$ in one corner of the grid. The robot starts at a given cell, must reach a goal cell, and must avoid obstacle cells.

Your task is to complete the file `solver.py` by implementing a Bounded Model Checking (BMC) encoding using the Z3 SMT solver. The solver must search for the shortest path from the start cell to the goal cell up to a given maximum bound.

2 Provided project

The project already contains the following components:

- `grid.py`: Defines the `GridProblem` data structure and loads grid instances from JSON files.
- `escape.py`: Command-line interface: loads a grid, calls `solve`, prints the result, validates the path, and optionally visualizes it.
- `manipulate.py`: Contains path-checking and visualization utilities.
- `solver.py`: Skeleton file to complete during the exam.
- `grids/`: Example grid instances for testing.

You must only modify `solver.py`. You may change helper function signatures if needed, but the public function `solve(problem: GridProblem, max_bound: int)` must keep the same name, meaning, and return type.

3 BE instructions

This section of the BE is open book (i.e. course and lab materials are allowed), but the **use of LLM is strictly prohibited**.

You must only upload `solver.py` on the Moodle deposit.

4 Grid model

A grid instance contains:

- a width W and height H ;
- an initial cell (x_s, y_s) ;
- a goal cell (x_g, y_g) ;
- a finite set of obstacle cells O .

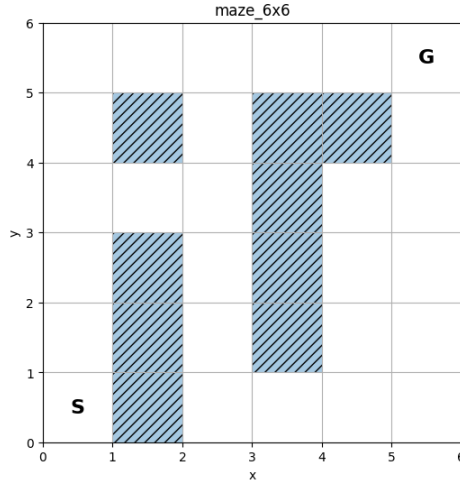


Figure 1: Example of a grid with a start cell (S), a goal cell (G), and obstacles (in blue).

A cell (x, y) is inside the grid iff: $0 \leq x < W$ and $0 \leq y < H$.

A cell is free iff it is inside the grid and is not an obstacle: $(x, y) \notin O$.

At each step, the robot may move exactly one cell in one of the four directions (left, right, up, down). Diagonal moves and staying in place are not allowed.

5 Expected behaviour

The function `solve(problem, max_bound)` must:

1. search for paths of length $0, 1, 2, \dots, \text{max_bound}$;
2. return the first satisfiable path found, so that the returned path is shortest within the explored bounds;
3. return `None` if no path exists up to `max_bound`.

If a path is found, it must be returned as a Python list of coordinate tuples:

```
[(0, 0), (1, 0), (2, 0), (2, 1)]
```

The first coordinate must be the initial position, and the last coordinate must be the goal position.

6 Running the program

You first need to create a virtual environment and install the Python dependencies:

```
python3 -m venv myEnv
source myEnv/bin/activate
pip3 install z3-solver matplotlib
```

Note that `matplotlib` library is only required for the `--viz` option.

You can run the project from the command line. Example commands:

```
python -m escape --grid grids/simple_4x4.json --max-bound 20
python -m escape --grid grids/open_8x5.json --max-bound 20
python -m escape --grid grids/maze_6x6.json --max-bound 20
python -m escape --grid grids/unsat_blocked.json --max-bound 20
```

To visualize a path, add the `--viz` option:

```
python -m escape --grid grids/simple_4x4.json --max-bound 20 --viz
```

7 Tasks

Task 1 – Create the BMC state variables

[1 pts]

Implement:

```
create_all_state_variables(max_bound: int)
```

For a bound k , the BMC encoding must represent all robot positions from step 0 to step k . You should create Z3 (state) variables to encode these steps.

Your implementation should make it easy to access the variables for others functions (e.g. by returning lists).

Task 2 – Encode the initial state

[1 pts]

Implement:

```
set_initial_state(solver, problem, x0, y0)
```

Add constraints forcing the first state to be equal to the initial cell.

Task 3 – Encode the transition relation

[2 pts]

Implement:

```
transition_relation(solver, problem, x_now, y_now, x_next, y_next)
```

For each transition, add constraints ensuring that:

- a) the next cell is inside the grid;
- b) the next cell is not an obstacle;
- c) the move is correct (left, right, up or down).

You can use Z3's `Abs`, `And`, `Or`, and `Not` operators.

Task 4 – Encode the goal state

[1 pts]

Implement:

```
set_goal(solver, problem, x, y)
```

Add constraints forcing the selected state to be equal to the goal cell.

Task 5 – Implement the BMC search

[4 pts]

Complete:

```
solve(problem: GridProblem, max_bound: int)
```

Your algorithm must satisfy the specification given in Sect. 5.

Task 6 – Lesson question

[1 pts]

Answer the following lesson question at the end of the file as Python comment:

In your own words, what is an NP-complete problem?

8 Example grid instances

Grid instance	Existence of a path	Shortest path length
simple_4x4.json	yes	6
open_8x5.json	yes	9
maze_6x6.json	yes	10
unsat_blocked.json	no	N/A